Diseño

Workshop #2

Spynet

Watching every part | Auditing every weakness

Grupo 5 — Los Extraditables

Juan Diego Rozo Álvarez	Backend
Santiago Alejandro Rojas Feo	Backend
Alejandro Medina Rojas	Backend
Misael Jesus Flores Anave	DevOps
Cristian David Arcia Quintero	Frontend
Marlon David Pabon Muñoz	CI/CD/Documentación

1. CRC Cards

1.1 Main Classes Selected

The following classes were selected because they directly participate in the core use case: analyzing a website URL and generating a complete technology profile. Classes that only provide utility, enforce structural contracts, or implement infrastructure concerns are excluded.

Class	Reason for Selection
Analyzer	Central orchestrator of the analysis pipeline. It is the single entry point that coordinates all detectors and services to produce the final result (US-11).
FrontendDetector	Identifies frontend frameworks such as React, Vue, Angular, and Next.js from HTML, scripts, and HTTP signals. Directly fulfills RF-02 and US-01.
BackendDetector	Identifies backend technologies such as Django, Laravel, WordPress, and Rails from HTTP headers, cookies, and HTML patterns. Directly fulfills RF-03 and US-01.
CDNDetector	Identifies CDN and hosting providers such as Cloudflare, Vercel, and Netlify using HTTP headers, DNS nameservers, and IP ranges. Fulfills RF-04.
WhoisService	Retrieves domain registration data including registrar, creation date, expiration date, and domain age. Fulfills RF-05 and US-06.
DnsRecordService	Resolves DNS records (A, MX, NS, TXT) for the analyzed domain. Its output feeds both CDNDetector and GeoService. Fulfills RF-06 and US-07.
GeoService	Translates the resolved IP address into geographic and network context: country, city, ISP, and organization. Fulfills RF-07 and US-08.
WaybackService	Retrieves historical snapshot data from the Wayback Machine CDX API, including snapshot count and the most recent timestamps. Fulfills RF-08 and US-09.

1.2 CRC Cards

Class: Analyzer	
Responsibilities	**Collaborators**
- Validate and normalize the input URL, adding the protocol prefix when absent. Perform a single shared HTTP request to fetch the target page and share the response among all detectors. - Coordinate the execution of all technology detectors with the shared page data. - Call domain intelligence services and pass resolved data to the appropriate detectors. - Consolidate results from all detectors and services into a single JSON response. - Handle individual service failures by setting the corresponding field to null and continuing the analysis. - Persist the completed analysis record to the database.	- FrontendDetector - BackendDetector - CDNDetector - DnsRecordService - WhoisService - GeoService - WaybackService

Class: FrontendDetector	
Responsibilities	**Collaborators**
- Inspect HTML content for framework-specific structures, attributes, and patterns. - Analyze script source URLs and inline JavaScript content for known library identifiers. - Match evidence against frontend technology signatures and assign a confidence score. - Return a list of detected technologies with name, category, confidence, and evidence.	- Analyzer (invokes detect()) - SignatureLoader (provides patterns via BaseDetector)

Class: BackendDetector

Responsibilities	Collaborators
- Examine HTTP response headers for server and technology disclosure signatures. - Inspect cookie names for framework-specific session identifiers. - Analyze HTML content for generator meta tags and CMS-specific path conventions. - Return a list of detected backend technologies with name, category, confidence, and evidence.	- Analyzer (invokes detect()) - SignatureLoader (provides patterns via BaseDetector)

Class: CDNDetector	
Responsibilities	**Collaborators**
- Inspect HTTP response headers for CDN-specific header names and values. - Match nameserver hostnames against k- nown CDN provider domain patterns. - Cross-reference the server IP against known IP ranges associated with hosting providers. - Return a list of identified CDN or hosting providers with confidence and evidence.	- Analyzer (invokes detect()) - DnsRecordService (provides nameserver list) - SignatureLoader (provides patterns via BaseDetector)

Class: WhoisService	
Responsibilities	**Collaborators**
- Query the WHOIS protocol for the analyzed domain and parse the raw response. - Extract registrar name, creation date, and expiration date from the WHOIS data. - Calculate the domain age in years from the creation date to the current date. - Return available fields when WHOIS privacy is active and set hidden fields to null without aborting the analysis.	- Analyzer (invokes get_whois())

Class: DnsRecordService	
Responsibilities	**Collaborators**

- Resolve A and AAAA records for the domain to obtain the server IP address. - Resolve MX, NS, and TXT records and return them in a structured format. - Supply the resolved IP address to GeoService for geolocation lookup. - Handle resolution failures gracefully and return available records without interrupting the analysis.	- Analyzer (invokes get_dns_records()) - CDNDetector (consumes nameserver list) - GeoService (consumes server IP)

Class: GeoService	
Responsibilities	**Collaborators**
- Accept the resolved server IP and query the ip-api.com geolocation endpoint. - Extract country, city, ISP, and organization from the API response. - Return null for the geo field when the external API is unavailable, without interrupting the analysis.	- Analyzer (invokes get_geoinfo()) - DnsRecordService (provides server IP)

Class: WaybackService	
Responsibilities	**Collaborators**
- Query the Wayback Machine CDX API for archived snapshots of the analyzed domain. - Count the total number of available snapshots for the domain. - Retrieve the timestamps and archive URLs of the most recent snapshots. - Return an empty result with snapshot count zero when no history is available, without generating an error.	- Analyzer (invokes get_wayback())

1.3 Excluded Classes

The following classes appear in the UML class diagram but were excluded from the CRC cards because they do not carry business responsibilities in the core use case. They serve structural, creational, or utility purposes.

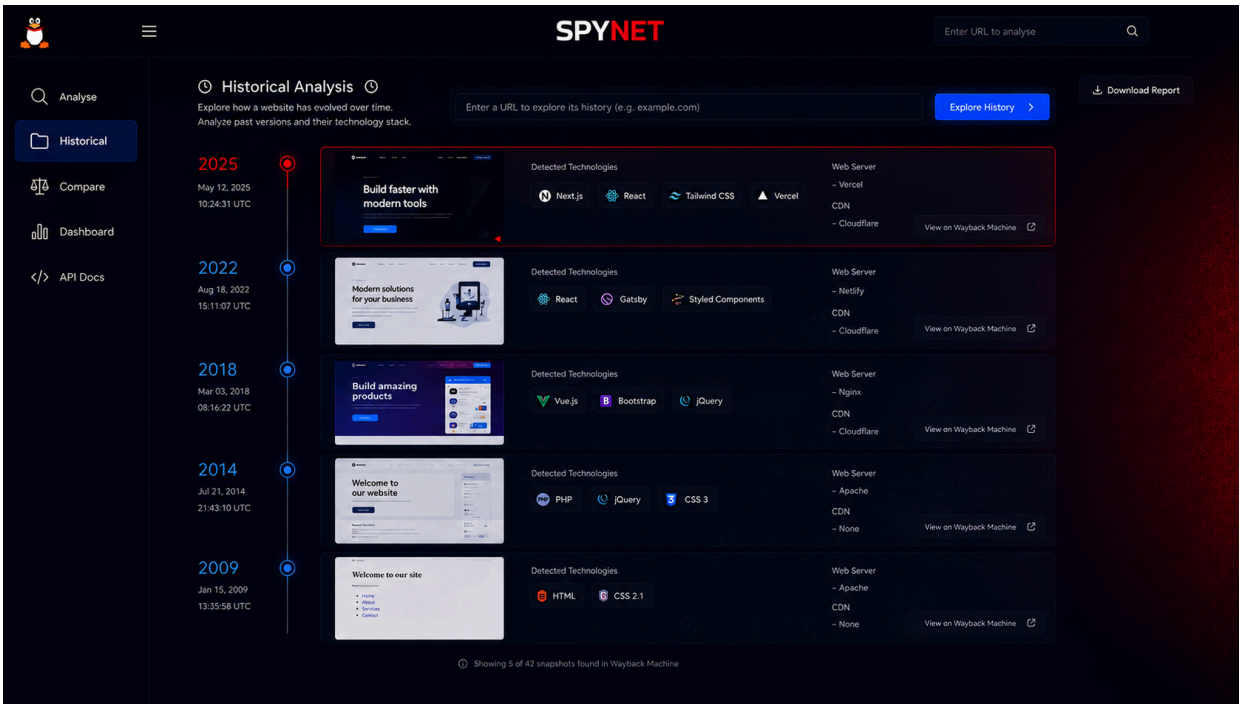
Class	Reason Excluded
BaseDetector	Abstract base class that provides shared helper methods to concrete detectors. It contains no business logic of its own and exists solely for code reuse.
BaseServices	Abstract base class that defines the fetch_service interface for all service classes. It enforces a structural contract but has no business behavior.
DetectorFactory	Implements the Factory pattern to instantiate detector objects by category. Its responsibility is purely creational and does not contribute to the analysis result.
SignatureLoader	Implements the Singleton pattern to load and cache technology signature files from disk. It is an infrastructure utility with no domain logic.
js_fetcher	A stateless utility module that fetches and prioritizes external JavaScript files. It contains helper functions, not class-level business responsibilities.

2. Mockups

- **Analyse view**

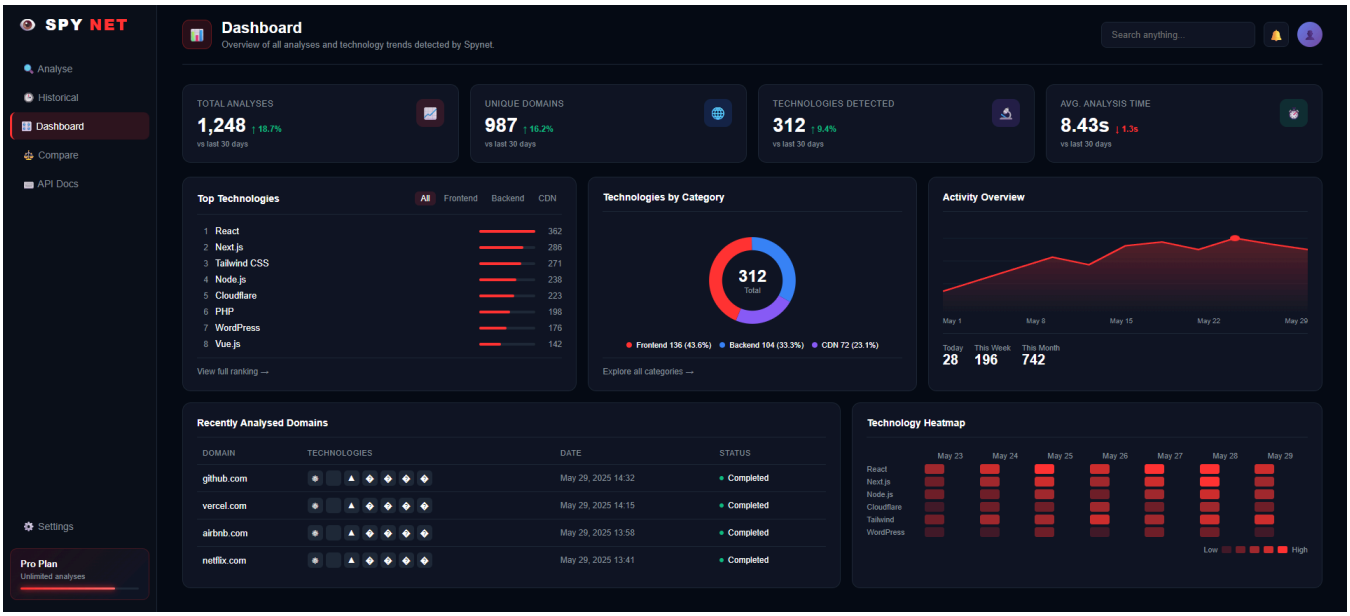
The Analysis Results screen displays the complete output of a URL scan. It is divided into six sections: DNS records showing IP address, nameservers, and mail exchangers; WHOIS data including creation date, expiration date, and registrar; a Technologies panel listing detected frameworks and libraries with their confidence level; a Geolocation card with country, city, ISP, and coordinates plotted on a map; a Summary card showing total technologies detected, average confidence score, and risk level; and a Snapshots section powered by the Wayback Machine that presents a visual timeline of historical captures, allowing users to track how the site's technology stack has evolved over time. An AI Analysis panel at the bottom provides a natural language summary of the findings.

- **Historical view**



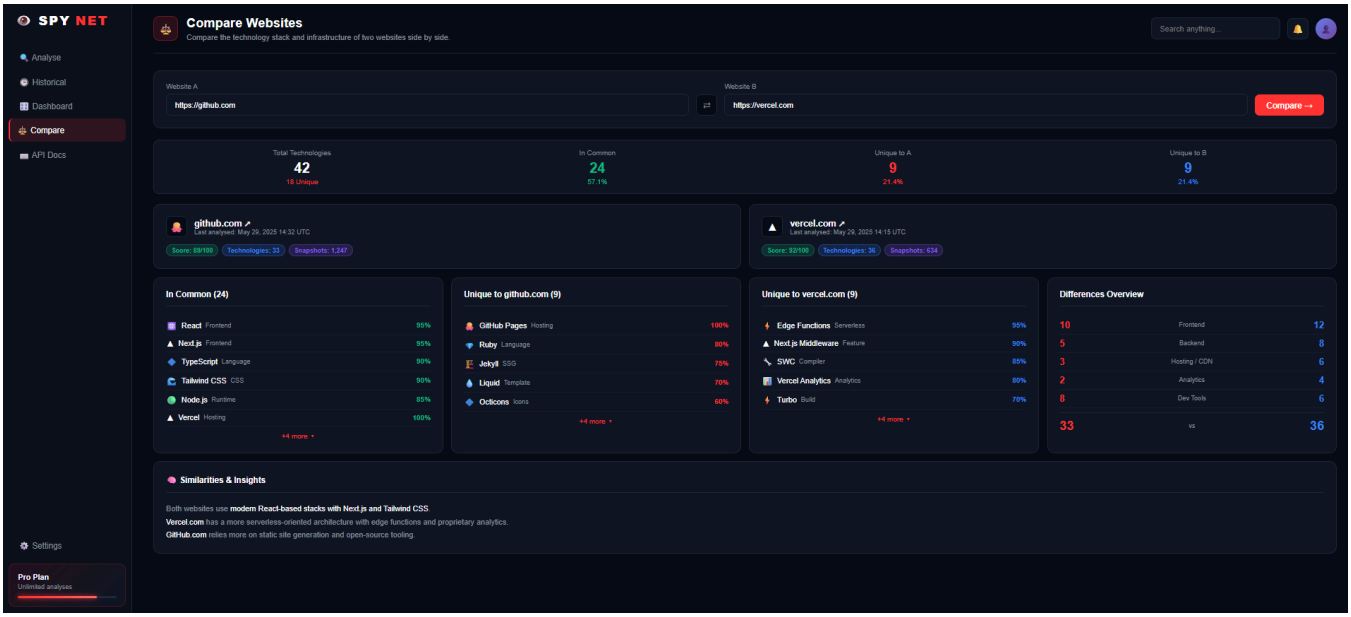
The Historical Analysis screen implements RF-19, allowing users to explore how a website's technology stack has evolved over time using Wayback Machine snapshots. It presents a vertical timeline where each entry corresponds to a historical capture, showing the snapshot year, exact date and time, a thumbnail preview of the site as it appeared at that moment, the technologies detected in that snapshot (frontend frameworks, CSS libraries), the web server, and the CDN in use. Each entry includes a direct link to view the original snapshot on the Wayback Machine. The view displays 5 of the total snapshots found, with the most recent at the top and the oldest at the bottom, illustrating the technological evolution of the site across years.

- **General Dashboard**



The Dashboard view functions as the central analytical hub, offering an aggregated and high-level overview of all scanned websites and macro technology trends. Across the top, a series of key performance indicators track metrics like total analyses, unique domains, total technologies detected, and the average analysis speed. The middle section visualizes global market trends using data charts, which include a ranking of top technologies like React and Next.js, a colorful ring chart breaking down technology categories, and an activity overview timeline. The bottom of the screen anchors the dashboard with a real-time list of recently analyzed domains alongside a technology heatmap that maps the frequency and concentration of specific framework usages over time.

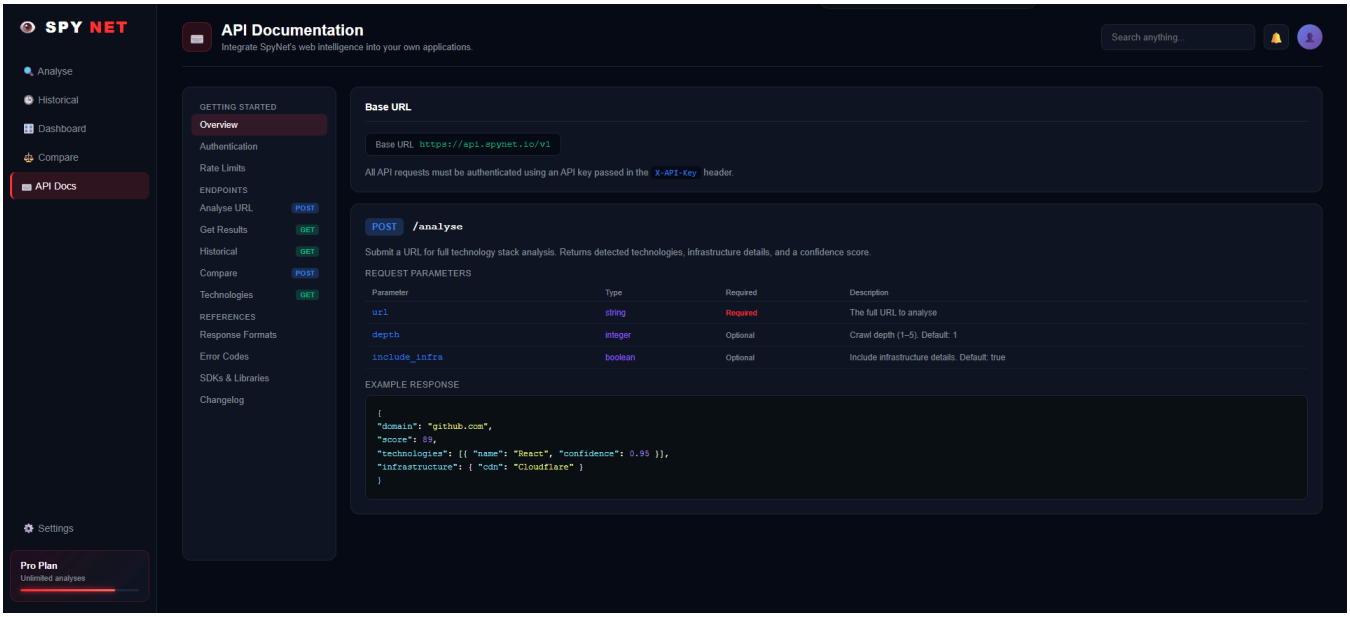
- **Comparison View**



The Compare Websites view provides side-by-side competitive intelligence by mapping the precise infrastructure differences between two specific domains. At the top, users can input two URLs to generate a high-level summary of shared and unique technologies, accompanied by

overall platform health and performance scores. The core of the interface uses structured lists to contrast exactly which frontend frameworks or deployment environments are held in common against the unique tools specific to each site. At the very bottom, an automated insights section translates this complex technical data into plain-language conclusions, explaining the underlying architectural differences between the two targets.

- **API Documentation View**

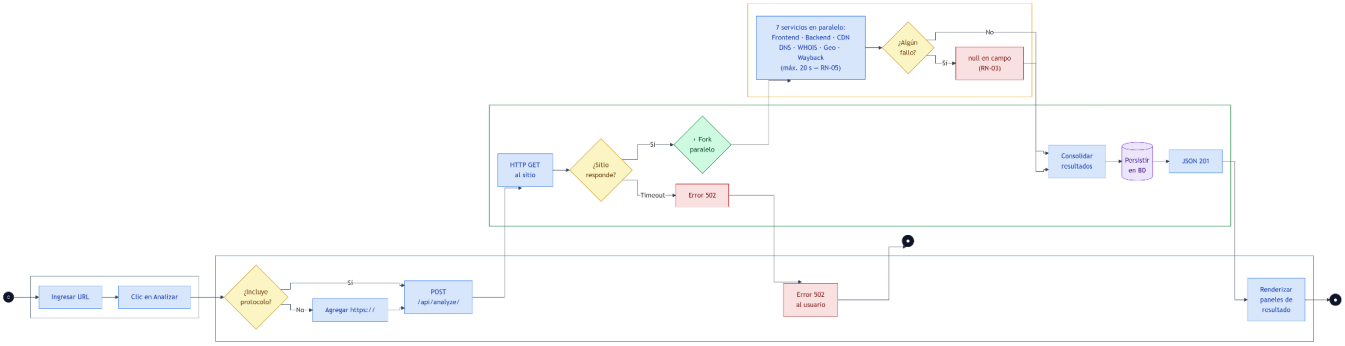


The API Documentation view serves as the developer hub designed to help users integrate SpyNet's web intelligence directly into their own applications. It displays the primary base URL alongside the required header authentication details for making requests. The interface outlines the technical specifications for endpoints like the POST /analyse function, detailing the required URL parameter alongside optional configurations like crawl depth and infrastructure inclusion. Finally, it provides an interactive JSON code snippet displaying an example response, which demonstrates exactly how the platform returns detected technology stacks, confidence scores, and content delivery network details to the user.

3. Business Model Processes

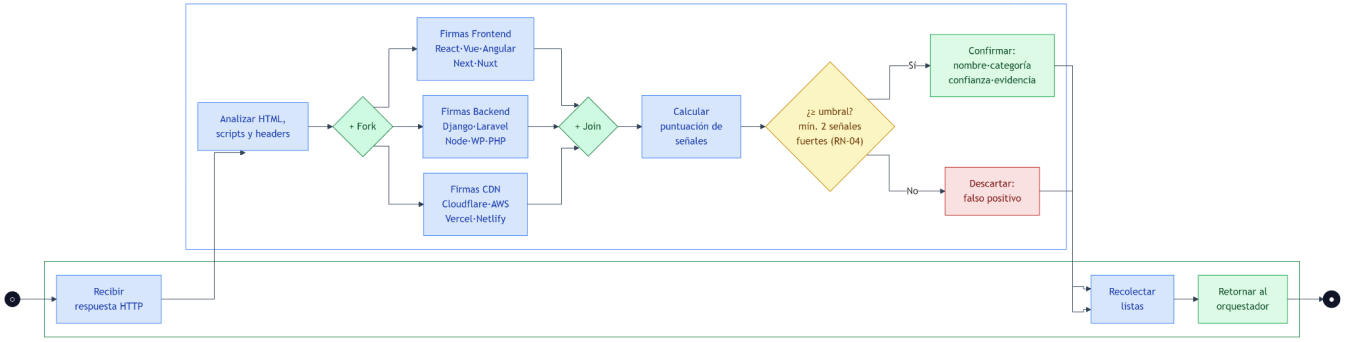
BP-01: Web analysis

This is the core user process. It begins when the user types a URL and clicks "Analizar." The frontend normalizes the URL if no protocol is present, then sends a POST to the backend. The analyzer facade makes one HTTP GET to the target site and, if it responds, shares that single response across all seven services (FrontendDetector, BackendDetector, CDNDetector, DNSService, WHOISService, GeoService, and WaybackService) which all run in parallel. If any individual service fails, its field returns null and the rest continue unaffected. The consolidated result is persisted in the database and returned as JSON 201 to the frontend, which renders it in categorized panels.



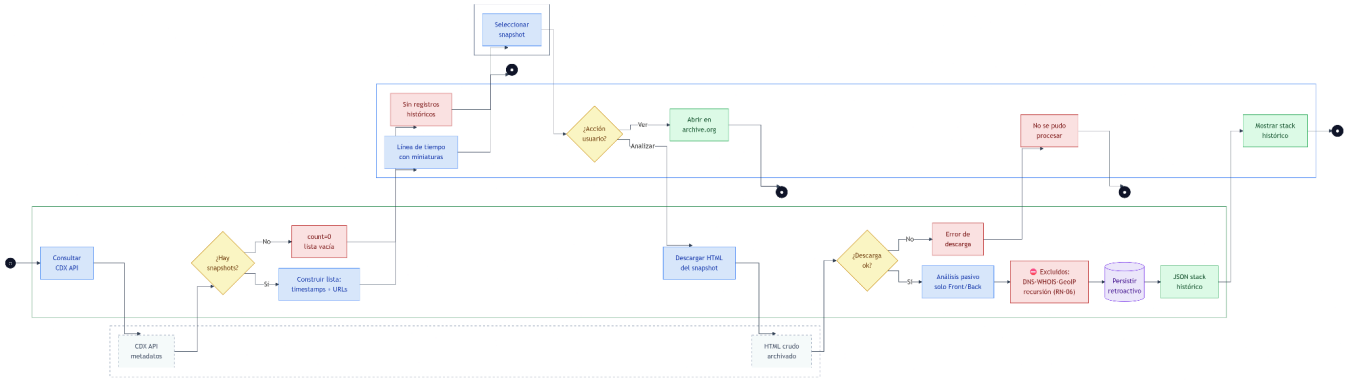
BP-02: Technology detection

This process models the anti-false-positive engine inside each detector. The facade distributes the HTTP response to three detectors running in parallel. Each detector loads its signature dictionary and scans HTML, script tags, and headers, accumulating a weighted score for each pattern it finds. A parallel AND Join then gathers all scores, and an XOR threshold gateway makes the final call: a technology is only confirmed and registered with its evidence if it meets the minimum confidence threshold. Patterns that don't reach the threshold are silently discarded, preventing noisy false positives from polluting the results.



BP-03: Historical Snapshot retroactive analysis

It begins automatically when a prior analysis is loaded. The backend queries the wayback machine CDX API to retrieve snapshot metadata. If no snapshots exist, the frontend shows a clean empty state. Otherwise, a chronological timeline with clickable thumbnails is rendered. The user can either open a snapshot directly in internet archive, or trigger a retroactive analysis: the backend downloads the raw archived HTML and runs a passive technology scan. Critically, the excluded node makes the architectural constraint explicit. No DNS, WHOIS, GeoIP queries, or recursive snapshot calls are allowed during this path. The result is stored as a new independent analysis record and displayed as a historical technology comparison.

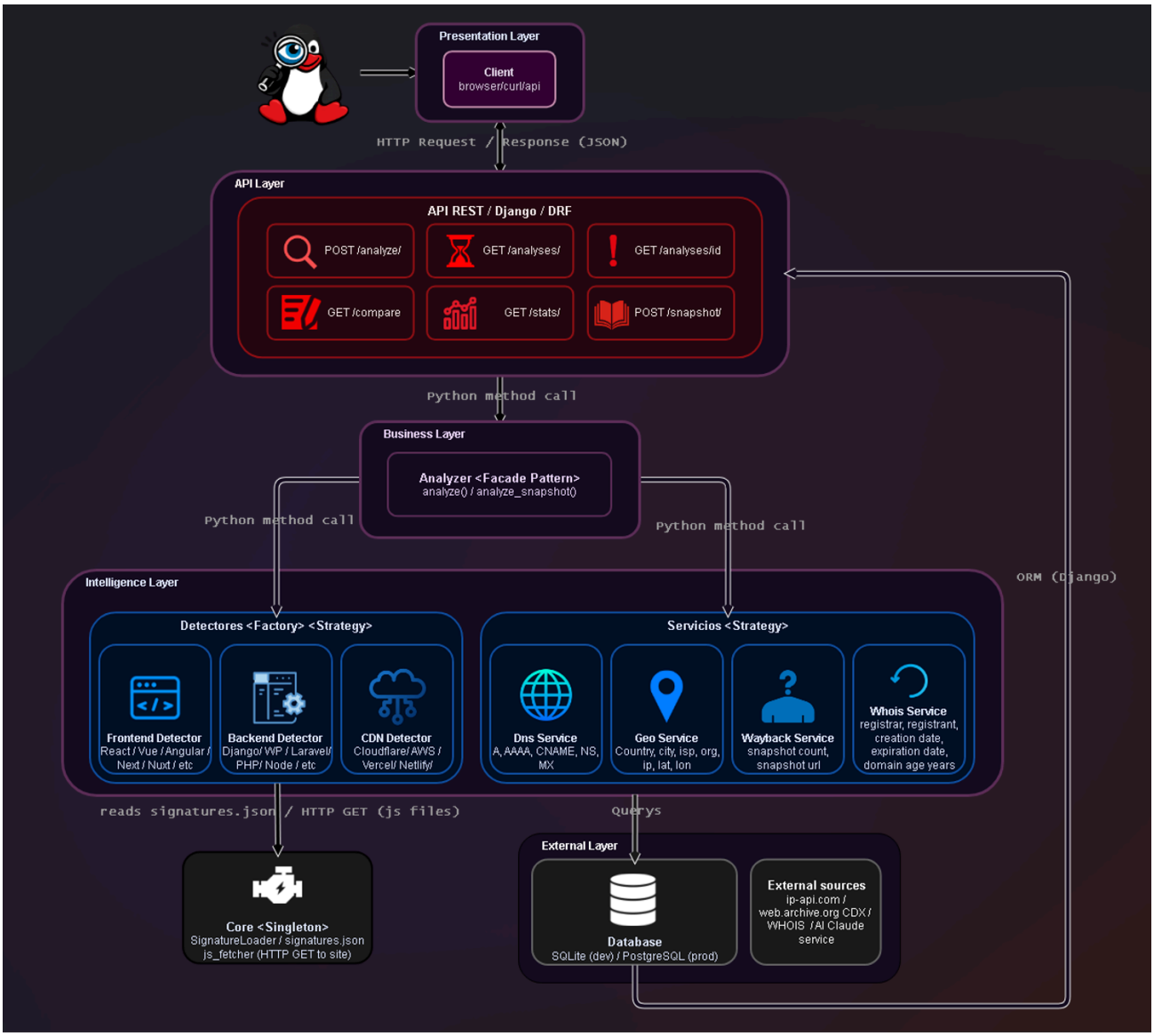


4. Architecture Diagram

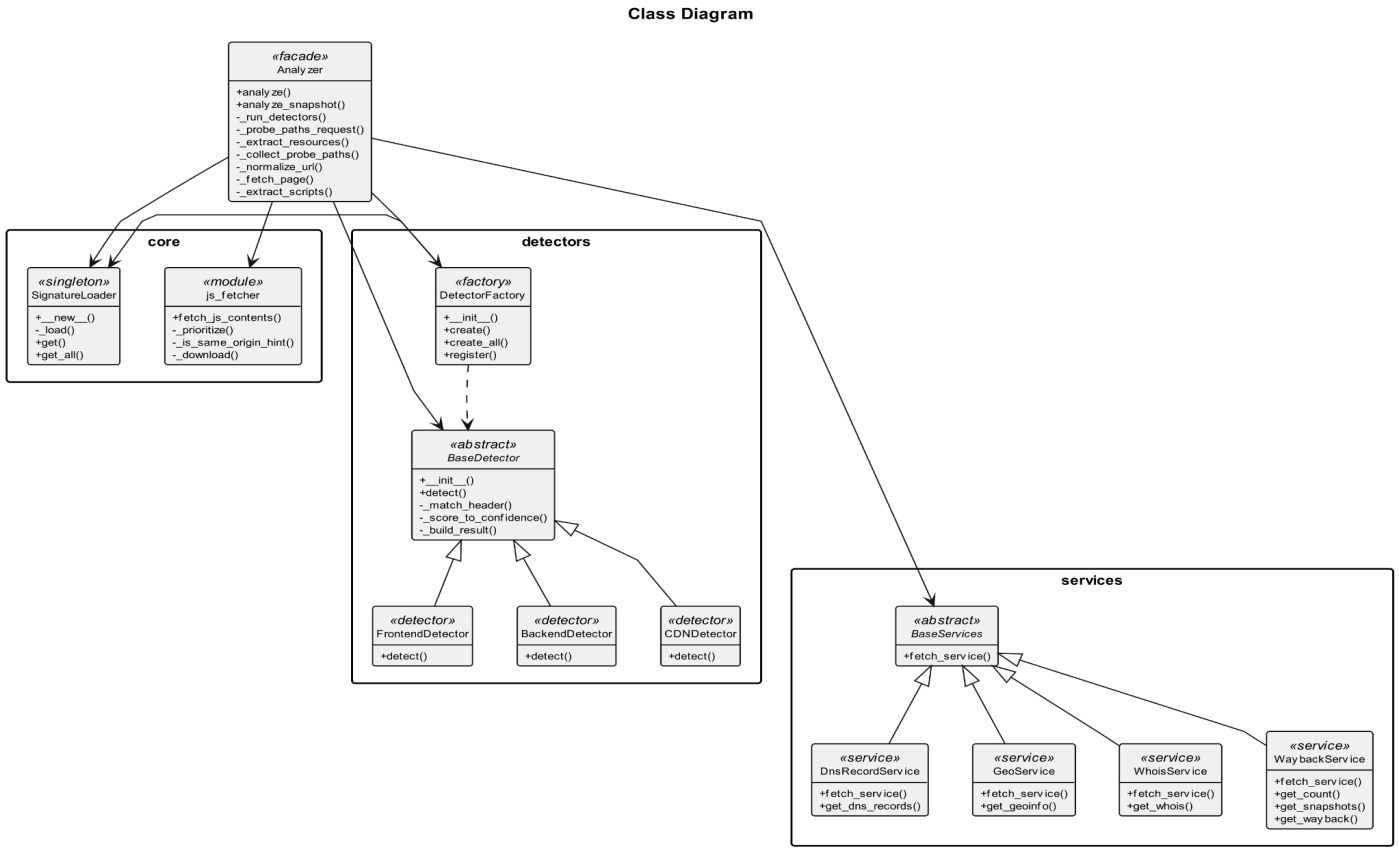
Spynet follows a layered client-server monolithic architecture. The client communicates via HTTP requests and receives JSON responses from the API Layer, built with Django and Django REST Framework, which exposes six endpoints covering analysis, history, comparison, statistics, and historical snapshot analysis.

Requests are forwarded through a Python method call to the Business Layer, where the Analyzer (Facade Pattern) acts as the single orchestration point. It delegates to two subsystems in the Intelligence Layer: the Detectors (Factory + Strategy), which identify frontend, backend, and CDN technologies through signature-based pattern matching; and the Services, which query external sources for DNS records, geolocation, WHOIS data, and Wayback Machine snapshots.

The Analyzer relies on the Core module (Singleton Pattern), which loads detection signatures and fetches JavaScript files for content analysis. Results are persisted to a relational database via Django ORM, while all external communication with ip-api.com, web.archive.org CDX, and WHOIS servers occurs at the External Layer.

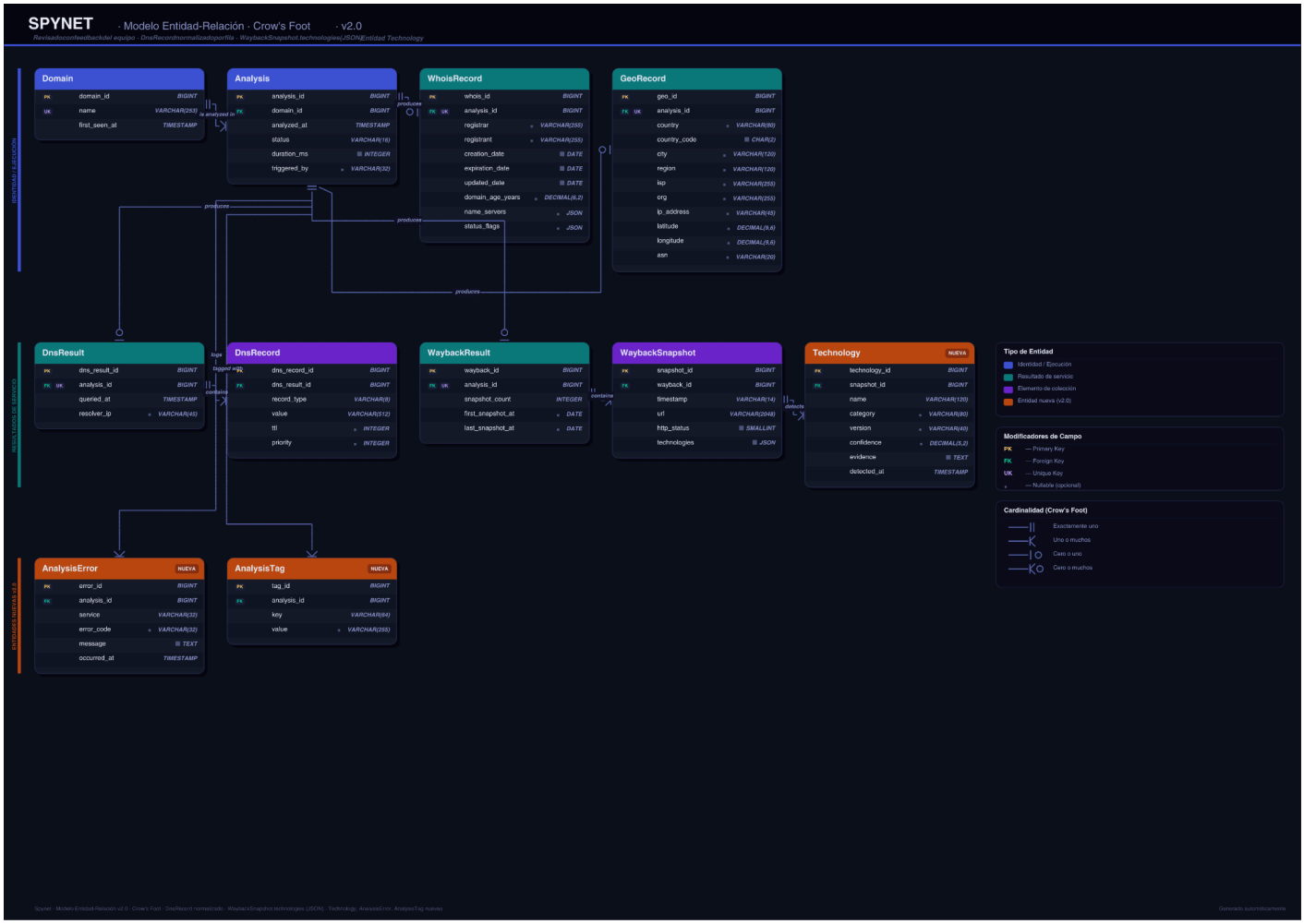

Presentation Layer
Client
browser/curl/api
HTTP Request / Response (JSON)
API Layer
API REST / Django / DRF
POST /analyze/
GET /analyses/
GET /analyses/id
GET /compare
GET /stats/
POST /snapshot/
Python method call
Business Layer
Analyzer <Facade Pattern>
analyze() / analyze_snapshot()
Python method call
Python method call
ORM (Django)
Intelligence Layer
Detectores <Factory> <Strategy>
Frontend Detector
React / Vue / Angular / Next / Nuxt / etc
Backend Detector
Django/ WP / Laravel/ PHP/ Node / etc
CDN Detector
Cloudflare/ AWS / Vercel/ Netlify/
Servicios <Strategy>
Dns Service
A, AAAA, CNAME, NS, MX
Geo Service
Country, city, isp, org, ip, lat, lon
Wayback Service
snapshot count, snapshot url
Whois Service
registrar, registrant, creation date, expiration date, domain age years
reads signatures.json / HTTP GET (js files)
Querys
Core <Singleton>
SignatureLoader / signatures.json
js_fetcher (HTTP GET to site)
External Layer
Database
SQLite (dev) / PostgreSQL (prod)
External sources
ip-api.com / web.archive.org CDX / WHOIS / AI Claude service

4. Class Diagram



5. Relational Database Model

The data model is centered around the Domain entity, which can have multiple Analysis records over time. Each analysis produces four service result entities: WhoisRecord, GeoRecord, DnsResult, and WaybackResult, all related one-to-one with the analysis. DNS records are normalized into individual DnsRecord rows grouped under a DnsResult. Each WaybackResult contains multiple WaybackSnapshot entries, and each snapshot stores its detected technologies in a Technology entity. Two additional entities support observability: AnalysisError logs per-service failures, and AnalysisTag allows flexible metadata tagging on any analysis.



References

JGraph Ltd. (2025). *draw.io* (versión 24) [Software de diagramación]. https://www.drawio.com

Microsoft. (2025). *Microsoft PowerPoint* (Microsoft 365) [Software de presentaciones]. https://www.microsoft.com/powerpoint